

SVM & Naive bayes

Theoretical -:

Question 1 - What is a Support Vector Machine (SVM)

A Support Vector Machine (SVM) is a supervised machine learning algorithm mainly used for classification tasks. It works by finding the best hyperplane that separates different classes with the maximum margin between them. The closest data points to the hyperplane are called support vectors, which determine the position of the boundary. For non-linear data, SVM uses the kernel trick to transform data into a higher dimension where it becomes easier to separate. Common kernels include linear, polynomial, and RBF. SVM is widely used because it is accurate and works well with complex datasets.

Question -2 What is the difference between Hard Margin and Soft Margin SVM

A Hard Margin SVM and Soft Margin SVM differ in how strictly they separate classes in a dataset. Hard Margin SVM is used when the data is perfectly linearly separable, meaning it finds a hyperplane that separates the classes with no misclassification at all. It does not allow any errors or overlapping points, which makes it very strict but sensitive to noise and outliers.

On the other hand, Soft Margin SVM is used when the data is not perfectly separable or contains noise and outliers. It allows some misclassifications by introducing a margin of error controlled by a parameter (C). This helps the model find a better balance between maximizing the margin and minimizing classification errors. Soft Margin SVM is more practical and widely used in real-world datasets because it handles noisy data more effectively.

Question - 3 What is the mathematical intuition behind SVM

The mathematical intuition behind Support Vector Machine (SVM) is to find the optimal hyperplane that separates two classes with the maximum margin. For a linear case, the hyperplane is defined as $w \cdot x + b = 0$, where w is the weight vector and b is the bias. SVM aims to choose w and b such that the distance between the hyperplane and the nearest data points of both classes (called support vectors) is maximized.

This is formulated as an optimization problem where we minimize $\|w\|^2 / 2$ subject to the constraint that all data points are correctly classified, i.e., $y_i(w \cdot x_i + b) \geq 1$. Minimizing $\|w\|^2$ increases the margin, which improves generalization. For non-separable data, SVM introduces slack variables to allow some misclassification, leading to the soft-margin formulation controlled by the parameter C , which balances margin size and classification error.

For non-linear data, SVM uses the kernel trick, which implicitly maps input data into a higher-dimensional space where a linear hyperplane can separate the classes. Thus,

mathematically, SVM is about convex optimization of margin maximization with constraints, ensuring the best separation between classes.

Question - 4 What is the role of Lagrange Multipliers in SVM

In Support Vector Machines (SVM), Lagrange Multipliers play a key role in solving the optimization problem of finding the maximum margin hyperplane under constraints. The main objective of SVM is to minimize the weight vector ($\|w\|^2$) while ensuring that all data points are correctly classified using the constraint ($y_i(w \cdot x_i + b) \geq 1$). Since this is a constrained optimization problem, Lagrange multipliers are introduced to convert it into an unconstrained form called the Lagrangian function, where each constraint is associated with a multiplier (α_i).

These Lagrange multipliers help identify which training points are important for defining the decision boundary. Only the points with non-zero multipliers become support vectors, meaning they lie closest to the hyperplane and directly influence its position. By solving the dual form of the optimization problem using Lagrange multipliers, SVM becomes computationally more efficient and allows the use of the kernel trick to handle non-linear data. Thus, Lagrange multipliers are essential because they transform the constrained optimization problem into a solvable form and help determine the critical support vectors that define the optimal hyperplane.

Question - 5 What are Support Vectors in SVM

Support vectors are the data points that lie closest to the decision boundary (hyperplane) in SVM. They are the most important points because they define the position and orientation of the hyperplane. The SVM model tries to maximize the margin between classes using these points. If support vectors change, the decision boundary also changes. Other points do not affect the model, only support vectors matter.

Question - 6 What is a Support Vector Classifier (SVC)

A Support Vector Classifier (SVC) is a type of Support Vector Machine (SVM) used specifically for classification problems. It works by finding the optimal hyperplane that separates different classes with the maximum margin. The closest points to the boundary, called support vectors, determine this hyperplane. SVC can handle both linear and non-linear data using different kernel functions like linear, polynomial, and RBF. It is widely used because it gives high accuracy and works well even on complex datasets.

Que - 7 What is a Support Vector Regressor (SVR)

A Support Vector Regressor (SVR) is a version of Support Vector Machine (SVM) used for regression problems. Instead of classifying data, it predicts continuous values. SVR tries to fit a line (or hyperplane) that keeps most data points within a certain error margin. Points outside this margin are called support vectors and influence the model. It also uses kernel functions to handle non-linear data. SVR is useful for accurate predictions in complex datasets.

Que - 8 What is the Kernel Trick in SVM

The Kernel Trick in SVM is a method used to handle non-linear data by transforming it into a higher-dimensional space where it becomes linearly separable. Instead of explicitly converting the data, the kernel function directly computes the similarity between data points in the new space, saving computation. Common kernels include linear, polynomial, and RBF (Gaussian). The kernel trick allows SVM to create complex decision boundaries efficiently without increasing computational cost too much.

Que- 9 Compare Linear Kernel, Polynomial Kernel, and RBF Kernel.

Linear Kernel: It is used when data is linearly separable, meaning classes can be separated with a straight line or hyperplane. It is simple, fast, and works well with high-dimensional data but cannot handle complex non-linear relationships.

Polynomial Kernel: It transforms data into a higher-dimensional space using a polynomial function. It can handle non-linear relationships, but performance depends on the degree of the polynomial. Higher degrees may lead to overfitting and higher computation cost.

RBF (Radial Basis Function) Kernel: It is the most commonly used kernel and works well for non-linear and complex data. It maps data into infinite dimensions using a Gaussian function. It gives strong performance but requires proper tuning of parameters like gamma.

Que - 10 What is the effect of the C parameter in SVM

The C parameter in SVM controls the trade-off between maximizing the margin and minimizing classification error. A high C value means the model tries to classify all training points correctly, leading to a smaller margin and possible overfitting. A low C value allows some misclassifications but creates a larger margin, improving generalization and reducing overfitting. So, C controls how strictly the model fits the training data.

Que - 11 What is the role of the Gamma parameter in RBF Kernel SVM

The Gamma parameter in RBF Kernel SVM controls how far the influence of a single training point reaches. A high gamma value means each point has a small influence, creating a very complex decision boundary and possibly leading to overfitting. A low gamma value means each point has a large influence, resulting in a smoother and simpler boundary, which may cause underfitting. So, gamma controls the flexibility of the decision boundary in RBF SVM.

Que - 12 What is the Naïve Bayes classifier, and why is it called "Naïve"

The Naïve Bayes classifier is a supervised machine learning algorithm based on Bayes' Theorem, used mainly for classification tasks. It predicts the probability of a class based on given features and assumes that all features are independent of each other.

It is called "Naïve" because it makes a strong and unrealistic assumption that all features are completely independent, which is rarely true in real-world data. Despite this simplification, it often works very well in practice, especially in text classification, spam detection, and sentiment analysis.

Que - 13 What is Bayes' Theorem

Bayes' Theorem is a mathematical formula used to find the probability of an event based on prior knowledge of related conditions. It describes how to update the probability of a hypothesis when new evidence is available.

It is given as:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

Here, $P(A|B)$ is the probability of event A given B has occurred, $P(B|A)$ is the probability of B given A is true, $P(A)$ is the prior probability of A, and $P(B)$ is the total probability of B.

In simple terms, Bayes' Theorem helps in updating beliefs based on new data, and it is the foundation of the Naïve Bayes classifier.

Que - 14 Explain the differences between Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes

Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes are three variants of the Naïve Bayes algorithm used for different types of data. **Gaussian Naïve Bayes** is used when the features are continuous and normally (Gaussian) distributed. It assumes that the data follows a bell-shaped curve and calculates probabilities using the Gaussian distribution, making it suitable for datasets like height, weight, or medical measurements.

Multinomial Naïve Bayes is used for discrete count data, especially in text classification problems like spam detection or sentiment analysis. It works by counting the frequency of features (such as word occurrences) and is commonly used in NLP tasks where word frequency matters.

Bernoulli Naïve Bayes is used for binary/boolean features, where features represent presence or absence (1 or 0). It is also used in text classification but focuses on whether a word appears or not, rather than how many times it appears. Thus, each variant is designed for a specific type of data, making Naïve Bayes flexible and efficient for different applications.

Que - 15 When should you use Gaussian Naïve Bayes over other variants

You should use **Gaussian Naïve Bayes** when your dataset has continuous numerical features that are assumed to follow a normal (Gaussian) distribution. It works best in cases where values like height, weight, temperature, or medical measurements are involved, because it models the probability using the mean and variance of each feature.

Unlike Multinomial or Bernoulli Naïve Bayes, which are designed for count data and binary data, Gaussian Naïve Bayes is suitable when features are real-valued and continuous. It is commonly used in problems like medical diagnosis and sensor data classification where measurements are continuous.

Que - 16 What are the key assumptions made by Naïve Bayes

The **Naïve Bayes classifier** makes two key assumptions. First, it assumes **feature independence**, meaning all input features are independent of each other given the class label. In simple terms, the presence or value of one feature does not affect another feature,

which is rarely true in real-world data. Second, it assumes that all features contribute equally and independently to the final outcome.

Another assumption depends on the variant used: Gaussian Naïve Bayes assumes that continuous features follow a normal distribution, while Multinomial and Bernoulli Naïve Bayes assume specific data types like counts or binary values. Despite these strong assumptions, Naïve Bayes still performs well in many practical applications like text classification.

Que - 17 What are the advantages and disadvantages of Naïve Bayes

Naïve Bayes has several advantages and disadvantages that affect its use in machine learning. One of its main advantages is that it is very simple and fast, making it suitable for large datasets and real-time predictions. It also performs well even with small training data and is highly effective for text classification problems like spam detection and sentiment analysis. Another advantage is that it handles high-dimensional data efficiently.

However, Naïve Bayes also has some disadvantages. Its biggest limitation is the strong assumption of feature independence, which is often not true in real-world data, leading to less accurate predictions in some cases. It also struggles with data where features are highly correlated. Additionally, it may perform poorly when the training data is not well distributed or does not match the assumed probability distribution.

Que - 18 Why is Naïve Bayes a good choice for text classification

Naïve Bayes is a good choice for text classification because it works very well with high-dimensional data like words in documents. In text data, each word is treated as a feature, and Naïve Bayes handles large numbers of features efficiently without requiring heavy computation. It also performs well even with small training datasets, which is common in text problems.

Another reason is that it works effectively with word frequency or presence-based features (like in spam detection or sentiment analysis). Models like Multinomial and Bernoulli Naïve Bayes are specifically designed for such data. Although it assumes feature independence (which is not fully true for text), it still gives good and fast results, making it a popular baseline model for NLP tasks.

Que - 19 Compare SVM and Naïve Bayes for classification tasks

SVM (Support Vector Machine) and **Naïve Bayes** are both classification algorithms but work in different ways. SVM finds the best hyperplane that separates classes with maximum margin, making it more powerful for complex and non-linear datasets (using kernels like RBF). It generally gives high accuracy, especially on medium and small datasets, but it is slower to train and requires parameter tuning like C and gamma.

On the other hand, Naïve Bayes is based on probability and Bayes' theorem, assuming that all features are independent. It is very fast, simple, and works well with high-dimensional data, especially text classification like spam detection. However, its accuracy may drop if the independence assumption is violated.

Que - 20 How does Laplace Smoothing help in Naïve Bayes

Laplace Smoothing helps in Naïve Bayes by handling the problem of **zero probability**. In text classification, if a word does not appear in the training data for a class, its probability becomes zero, which makes the entire prediction zero for that class.

Laplace smoothing solves this by adding a small value (usually 1) to all counts, so no probability becomes zero. This ensures that unseen words still get a small probability instead of completely breaking the model. As a result, the model becomes more stable and performs better on new or unseen data.

Practical -:

Que - 21 Write a Python program to train an SVM Classifier on the Iris dataset and evaluate accuracy

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

```
model = SVC(kernel='rbf')
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Output -

Accuracy: 1.0

Que - 22 Write a Python program to train two SVM classifiers with Linear and RBF kernels on the Wine dataset, then compare their accuracies

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
wine = load_wine()
X = wine.data
y = wine.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

```
linear_model = SVC(kernel='linear')
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
linear_acc = accuracy_score(y_test, linear_pred)
```

```
rbf_model = SVC(kernel='rbf')
rbf_model.fit(X_train, y_train)
rbf_pred = rbf_model.predict(X_test)
rbf_acc = accuracy_score(y_test, rbf_pred)
```

```
print("Linear SVM Accuracy:", linear_acc)
print("RBF SVM Accuracy:", rbf_acc)
```

Outputs -

```
Linear SVM Accuracy: 1.0
RBF SVM Accuracy: 0.8055555555555556
```

Que - 23 Write a Python program to train an SVM Classifier with a Polynomial Kernel and visualize the decision boundary

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.svm import SVC
```

```
from sklearn.datasets import make_moons
```

```
X, y = make_moons(n_samples=200, noise=0.2, random_state=42)
```

```
model = SVC(kernel='poly', degree=3)
```

```
model.fit(X, y)
```

```
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
```

```
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(  
    np.arange(x_min, x_max, 0.05),  
    np.arange(y_min, y_max, 0.05)  
)
```

```
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
```

```
Z = Z.reshape(xx.shape)
```

```
plt.contourf(xx, yy, Z, alpha=0.3)
```

```
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k')
```

```
plt.title("SVM with Polynomial Kernel")
```

```
plt.show()
```

Que - 24 Write a Python program to train a Gaussian Naïve Bayes classifier on the Breast Cancer dataset and evaluate accuracy

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB  
from sklearn.metrics import accuracy_score
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Output -

```
Accuracy: 0.9736842105263158
```

Que - 25 Write a Python program to train a Multinomial Naïve Bayes classifier for text classification using the 20 Newsgroups dataset

```
from sklearn.datasets import fetch_20newsgroups
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
newsgroups = fetch_20newsgroups(subset='all')
```

```
X = newsgroups.data
```

```
y = newsgroups.target
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=1)
```

```
vectorizer = CountVectorizer(stop_words='english')
```

```
X_train_vec = vectorizer.fit_transform(X_train)
```

```
X_test_vec = vectorizer.transform(X_test)
```

```
model = MultinomialNB()
```

```
model.fit(X_train_vec, y_train)
```

```
y_pred = model.predict(X_test_vec)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred))
```

Output -

```
Accuracy: 0.8745358090185676
```

```
Classification Report:
```

```
precision    recall  f1-score   support
```

0	0.86	0.91	0.88	151
1	0.67	0.92	0.78	202
2	0.96	0.35	0.52	195
3	0.61	0.86	0.71	183
4	0.89	0.90	0.90	205
5	0.85	0.85	0.85	215
6	0.93	0.70	0.80	193
7	0.90	0.95	0.93	196
8	0.95	0.94	0.95	168
9	0.98	0.96	0.97	211
10	0.95	0.97	0.96	198
11	0.92	0.96	0.94	201
12	0.92	0.83	0.87	202
13	0.95	0.95	0.95	194
14	0.92	0.97	0.95	189
15	0.88	0.99	0.93	202
16	0.89	0.93	0.91	188
17	0.95	1.00	0.97	182
18	0.84	0.89	0.87	159
19	0.95	0.58	0.72	136
accuracy				0.87 3770
macro avg		0.89 0.87 0.87	3770	
weighted avg		0.89 0.87 0.87	3770	

Que - 26 Write a Python program to train an SVM Classifier with different C values and compare the decision boundaries visually

```
import numpy as np
```

```
import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.svm import SVC

# Load Iris dataset (use only 2 features for visualization)

iris = datasets.load_iris()

X = iris.data[:, :2]

y = iris.target

# Different C values

C_values = [0.1, 1, 100]

# Create mesh grid

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1

y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

# Plot decision boundaries

plt.figure(figsize=(15, 4))

for i, C in enumerate(C_values):

    # Train SVM

    model = SVC(kernel='linear', C=C)

    model.fit(X, y)
```

```
# Predict on mesh grid

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])

Z = Z.reshape(xx.shape)
```

```
# Plot

plt.subplot(1, 3, i+1)

plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(X[:, 0], X[:, 1], c=y)

plt.title(f"SVM Decision Boundary (C={C})")
```

```
plt.show()
```

Que - 27 Write a Python program to train a Bernoulli Naïve Bayes classifier for binary classification on a dataset with binary features

```
from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import BernoulliNB

from sklearn.metrics import accuracy_score, classification_report
```

```
# Binary feature dataset
```

```
# Features: [Feature1, Feature2, Feature3]
```

```
X = [

    [1, 0, 1],

    [1, 1, 0],

    [0, 1, 1],

    [0, 0, 1],

    [1, 1, 1],

    [0, 0, 0],

    [1, 0, 0],
```

```

    [0, 1, 0]
]

# Binary target labels
y = [1, 1, 0, 0, 1, 0, 1, 0]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Bernoulli Naive Bayes model
model = BernoulliNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

```

Output -

```
Predictions: [1 0]
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Que - 28 Write a Python program to apply feature scaling before training an SVM model and compare results with unscaled data

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()

X = data.data

y = data.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# ----- Without Scaling -----

model_unscaled = SVC(kernel='rbf')
model_unscaled.fit(X_train, y_train)

y_pred_unscaled = model_unscaled.predict(X_test)
acc_unscaled = accuracy_score(y_test, y_pred_unscaled)

# ----- With Scaling -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_scaled = SVC(kernel='rbf')
model_scaled.fit(X_train_scaled, y_train)

y_pred_scaled = model_scaled.predict(X_test_scaled)
acc_scaled = accuracy_score(y_test, y_pred_scaled)

# Compare Results

print("Accuracy without scaling:", acc_unscaled)
print("Accuracy with scaling:", acc_scaled)

```

Output -

Accuracy without scaling: 1.0

Accuracy with scaling: 1.0

Que - 29 Write a Python program to train a Gaussian Naïve Bayes model and compare the predictions before and after Laplace Smoothing

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()

X = data.data

y = data.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# ----- Default GaussianNB -----

model1 = GaussianNB() # default smoothing

model1.fit(X_train, y_train)


y_pred1 = model1.predict(X_test)


# ----- GaussianNB with higher smoothing -----

model2 = GaussianNB(var_smoothing=1e-2)
```

```
model2.fit(X_train, y_train)
```

```
y_pred2 = model2.predict(X_test)
```

```
# Compare
```

```
print("Predictions with default GaussianNB:")
```

```
print(y_pred1)
```

```
print("\nPredictions with higher smoothing:")
```

```
print(y_pred2)
```

```
print("\nAccuracy with default GaussianNB:",
```

```
      accuracy_score(y_test, y_pred1))
```

```
print("Accuracy with higher smoothing:",
```

```
      accuracy_score(y_test, y_pred2))
```

Output -

```
Predictions with default GaussianNB:
```

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

```
Predictions with higher smoothing:
```

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

```
Accuracy with default GaussianNB: 1.0
```

```
Accuracy with higher smoothing: 1.0
```

Que - 30 Write a Python program to train an SVM Classifier and use GridSearchCV to tune the hyperparameters (C, gamma, kernel)

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# Define SVM model

svm = SVC()


# Hyperparameter grid

param_grid = {

    'C': [0.1, 1, 10, 100],

    'gamma': [1, 0.1, 0.01, 0.001],

    'kernel': ['linear', 'rbf']

}


# GridSearchCV
```

```

grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)

# Train model
grid_search.fit(X_train, y_train)

# Best parameters
print("Best Parameters:", grid_search.best_params_)

# Best model
best_model = grid_search.best_estimator_

# Predictions
y_pred = best_model.predict(X_test)

# Accuracy
print("Test Accuracy:", accuracy_score(y_test, y_pred))

```

Output -

```
Best Parameters: {'C': 1, 'gamma': 1, 'kernel': 'linear'}
```

```
Test Accuracy: 1.0
```

Que - 31 Write a Python program to train an SVM Classifier on an imbalanced dataset and apply class weighting and check it improve accuracy

```
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report


# Create imbalanced dataset

X, y = make_classification(

    n_samples=2000,

    n_features=10,

    n_classes=2,

    weights=[0.9, 0.1], # imbalanced (90% vs 10%)

    random_state=42

)


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)


model1 = SVC(kernel='rbf', random_state=42)

model1.fit(X_train, y_train)


y_pred1 = model1.predict(X_test)


print("WITHOUT class weighting")

print("Accuracy:", accuracy_score(y_test, y_pred1))
```

```
model2 = SVC(kernel='rbf', class_weight='balanced', random_state=42)
```

```
model2.fit(X_train, y_train)
```

```
y_pred2 = model2.predict(X_test)
```

```
print("\nWITH class weighting")
```

```
print("Accuracy:", accuracy_score(y_test, y_pred2))
```

Output -

```
WITHOUT class weighting
```

```
Accuracy: 0.9425
```

```
WITH class weighting
```

```
Accuracy: 0.8975
```

Que - 32 Write a Python program to implement a Naïve Bayes classifier for spam detection using email data

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.metrics import accuracy_score
```

```
# Dataset
```

```
emails = [
```

```
    "Win free iPhone now",
```

```
    "Meeting scheduled tomorrow",
```

```
    "Claim your free prize",
```

```
"Please send report",
"Earn money fast click here",
"Project discussion today"
]

labels = [1, 0, 1, 0, 1, 0] # 1=spam, 0=ham

# Split
X_train, X_test, y_train, y_test = train_test_split(
    emails, labels, test_size=0.3, random_state=42
)

# Vectorization
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)

# Model
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# New prediction
```

```
print("Prediction:", model.predict(vectorizer.transform(["Win cash now"])))
```

Output -

```
Accuracy: 1.0
```

```
Prediction: [0]
```

Que - 33 Write a Python program to train an SVM Classifier and a Naïve Bayes Classifier on the same dataset and compare their accuracy

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
svm_model = SVC(kernel='rbf')
```

```
svm_model.fit(X_train, y_train)
```

```
svm_pred = svm_model.predict(X_test)
```

```
svm_acc = accuracy_score(y_test, svm_pred)
```

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_pred = nb_model.predict(X_test)
nb_acc = accuracy_score(y_test, nb_pred)
```

```
print("SVM Accuracy:", svm_acc)
```

```
print("Naïve Bayes Accuracy:", nb_acc)
```

Output -

```
SVM Accuracy: 1.0
```

```
Naïve Bayes Accuracy: 1.0
```

Que -34 Write a Python program to perform feature selection before training a Naïve Bayes classifier and compare results

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

)

```
model1 = GaussianNB()
```

```
model1.fit(X_train, y_train)
```

```
pred1 = model1.predict(X_test)
```

```
acc1 = accuracy_score(y_test, pred1)
```

```
selector = SelectKBest(score_func=chi2, k=2)
```

```
X_train_sel = selector.fit_transform(X_train, y_train)
```

```
X_test_sel = selector.transform(X_test)
```

```
model2 = GaussianNB()
```

```
model2.fit(X_train_sel, y_train)
```

```
pred2 = model2.predict(X_test_sel)
```

```
acc2 = accuracy_score(y_test, pred2)
```

```
print("Accuracy without feature selection:", acc1)
```

```
print("Accuracy with feature selection:", acc2)
```

Output -

```
Accuracy without feature selection: 1.0
```

```
Accuracy with feature selection: 1.0
```

Que - 35 Write a Python program to train an SVM Classifier using One-vs-Rest (OvR) and One-vs-One (OvO) strategies on the Wine dataset and compare their accuracy

```
from sklearn.datasets import load_wine
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
```

```
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier

from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_wine()
```

```
X = data.data
```

```
y = data.target
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
ovr_model = OneVsRestClassifier(SVC(kernel='linear'))
```

```
ovr_model.fit(X_train, y_train)
```

```
ovr_pred = ovr_model.predict(X_test)
```

```
ovr_acc = accuracy_score(y_test, ovr_pred)
```

```
ovo_model = OneVsOneClassifier(SVC(kernel='linear'))
```

```
ovo_model.fit(X_train, y_train)
```

```
ovo_pred = ovo_model.predict(X_test)
```

```
ovo_acc = accuracy_score(y_test, ovo_pred)
```

```
print("OvR Accuracy:", ovr_acc)
```

```
print("OvO Accuracy:", ovo_acc)
```

Output -

OvR Accuracy: 1.0

OvO Accuracy: 1.0

Que - 36 Write a Python program to train an SVM Classifier using Linear, Polynomial, and RBF kernels on the Breast Cancer dataset and compare their accuracy

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


model_linear = SVC(kernel='linear')

model_linear.fit(X_train, y_train)

pred_linear = model_linear.predict(X_test)

acc_linear = accuracy_score(y_test, pred_linear)


model_poly = SVC(kernel='poly', degree=3)

model_poly.fit(X_train, y_train)

pred_poly = model_poly.predict(X_test)
```

```
acc_poly = accuracy_score(y_test, pred_poly)
```

```
model_rbf = SVC(kernel='rbf')
```

```
model_rbf.fit(X_train, y_train)
```

```
pred_rbf = model_rbf.predict(X_test)
```

```
acc_rbf = accuracy_score(y_test, pred_rbf)
```

```
print("Linear Kernel Accuracy:", acc_linear)
```

```
print("Polynomial Kernel Accuracy:", acc_poly)
```

```
print("RBF Kernel Accuracy:", acc_rbf)
```

Output -

```
Linear Kernel Accuracy: 0.956140350877193
```

```
Polynomial Kernel Accuracy: 0.9473684210526315
```

```
RBF Kernel Accuracy: 0.9473684210526315
```

Que - 37 Write a Python program to train an SVM Classifier using Stratified K-Fold Cross-Validation and compute the average accuracy

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.svm import SVC
```

```
from sklearn.model_selection import StratifiedKFold
```

```
from sklearn.metrics import accuracy_score
```

```
import numpy as np
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```

# Stratified K-Fold (preserves class distribution)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

model = SVC(kernel='rbf')

accuracies = []

# Cross-validation loop
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

```

```

# Average accuracy

print("Fold Accuracies:", accuracies)

print("Average Accuracy:", np.mean(accuracies))

```

Output -

```

Fold Accuracies: [0.9385964912280702, 0.8771929824561403,
0.8947368421052632, 0.9385964912280702, 0.9203539823008849]

```

```

Average Accuracy: 0.9138953578636858

```

Que - 38 Write a Python program to train a Naïve Bayes classifier using different prior probabilities and compare performance

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


model1 = GaussianNB()

model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

acc1 = accuracy_score(y_test, pred1)


model2 = GaussianNB(priors=[0.33, 0.33, 0.34])

model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

acc2 = accuracy_score(y_test, pred2)


model3 = GaussianNB(priors=[0.7, 0.2, 0.1])
```

```
model3.fit(X_train, y_train)

pred3 = model3.predict(X_test)

acc3 = accuracy_score(y_test, pred3)


print("Accuracy (default priors):", acc1)

print("Accuracy (balanced priors):", acc2)

print("Accuracy (skewed priors):", acc3)
```

Output -

```
Accuracy (default priors): 1.0

Accuracy (balanced priors): 1.0

Accuracy (skewed priors): 1.0
```

Que - 39 Write a Python program to perform Recursive Feature Elimination (RFE) before training an SVM Classifier and compare accuracy

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.feature_selection import RFE

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42
```

```

)

model1 = SVC(kernel='rbf')

model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

acc1 = accuracy_score(y_test, pred1)


svm_estimator = SVC(kernel='linear') # RFE needs linear estimator


rfe = RFE(estimator=svm_estimator, n_features_to_select=10)

X_train_rfe = rfe.fit_transform(X_train, y_train)

X_test_rfe = rfe.transform(X_test)


model2 = SVC(kernel='rbf')

model2.fit(X_train_rfe, y_train)

pred2 = model2.predict(X_test_rfe)

acc2 = accuracy_score(y_test, pred2)


print("Accuracy without RFE:", acc1)

print("Accuracy with RFE:", acc2)

```

Output -

```
Accuracy without RFE: 0.9473684210526315
```

```
Accuracy with RFE: 0.9473684210526315
```

Que - 40 Write a Python program to train an SVM Classifier and evaluate its performance using Precision, Recall, and F1-Score instead of accuracy

```

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

```

```
from sklearn.svm import SVC

from sklearn.metrics import classification_report


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# Train SVM classifier

model = SVC(kernel='rbf')

model.fit(X_train, y_train)


# Predictions

y_pred = model.predict(X_test)


# Evaluation using Precision, Recall, F1-score

print("Classification Report:\n")

print(classification_report(y_test, y_pred))
```

Output -

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.86	0.93	43
1	0.92	1.00	0.96	71
accuracy			0.95	114
macro avg	0.96	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114

Que - 41 Write a Python program to train a Naïve Bayes Classifier and evaluate its performance using Log Loss (Cross-Entropy Loss)

```

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import log_loss


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# Train Naïve Bayes model

model = GaussianNB()

```

```
model.fit(X_train, y_train)

# Predict probabilities (important for log loss)
y_prob = model.predict_proba(X_test)

# Compute Log Loss (Cross-Entropy Loss)
loss = log_loss(y_test, y_prob)

print("Log Loss:", loss)
```

Output -

```
Log Loss: 0.2037342715214191
```

Que - 42 Write a Python program to train an SVM Classifier and visualize the Confusion Matrix using seaborn

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import confusion_matrix, classification_report

import seaborn as sns

import matplotlib.pyplot as plt

# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# Train SVM classifier
model = SVC(kernel='rbf')
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plot using seaborn
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - SVM Classifier")
plt.show()

# Optional: Classification report
print(classification_report(y_test, y_pred))

```

Output -

	precision	recall	f1-score	support
0	1.00	0.86	0.93	43

1	0.92	1.00	0.96	71
accuracy			0.95	114
macro avg	0.96	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114

Que - 43 Write a Python program to train an SVM Regressor (SVR) and evaluate its performance using Mean Absolute Error (MAE) instead of MSE

```

from sklearn.datasets import make_regression

from sklearn.model_selection import train_test_split

from sklearn.svm import SVR

from sklearn.metrics import mean_absolute_error

from sklearn.preprocessing import StandardScaler

# Create regression dataset

X, y = make_regression(n_samples=1000, n_features=5, noise=10, random_state=42)

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature scaling (important for SVR)

scaler_X = StandardScaler()

scaler_y = StandardScaler()

X_train = scaler_X.fit_transform(X_train)

X_test = scaler_X.transform(X_test)

```

```

y_train = scaler_y.fit_transform(y_train.reshape(-1, 1)).ravel()

# Train SVR model

model = SVR(kernel='rbf')

model.fit(X_train, y_train)

# Predictions

y_pred = model.predict(X_test)

# Convert back to original scale

y_pred = scaler_y.inverse_transform(y_pred.reshape(-1, 1)).ravel()

y_test_orig = scaler_y.inverse_transform(
    scaler_y.transform(y_test.reshape(-1, 1))
).ravel()

# Evaluate using MAE

mae = mean_absolute_error(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)

```

Output -

```

Mean Absolute Error (MAE): 10.549641840847812

```

Que - 44 Write a Python program to train a Naïve Bayes classifier and evaluate its performance using the ROC-AUC score

```

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import roc_auc_score

```

```

# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# Train Naïve Bayes model

model = GaussianNB()

model.fit(X_train, y_train)


# Predict probabilities (important for ROC-AUC)

y_prob = model.predict_proba(X_test)[:, 1]


# ROC-AUC score

auc = roc_auc_score(y_test, y_prob)


print("ROC-AUC Score:", auc)

```

Output -

```
ROC-AUC Score: 0.9983622666229938
```

Que - 45 Write a Python program to train an SVM Classifier and visualize the Precision-Recall Curve

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import precision_recall_curve, average_precision_score

import matplotlib.pyplot as plt


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# Train SVM model (probability=True needed for curves)

model = SVC(kernel='rbf', probability=True)

model.fit(X_train, y_train)


# Get probability scores

y_scores = model.predict_proba(X_test)[:, 1]


# Precision-Recall values

precision, recall, _ = precision_recall_curve(y_test, y_scores)


# Average Precision Score
```

```
ap_score = average_precision_score(y_test, y_scores)
```

```
# Plot PR Curve
```

```
plt.figure()
```

```
plt.plot(recall, precision, label=f"AP = {ap_score:.2f}")
```

```
plt.xlabel("Recall")
```

```
plt.ylabel("Precision")
```

```
plt.title("Precision-Recall Curve (SVM)")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```